PROJECT 02

AI 에이전트 협업 개발

차곡 | 미수금 · 입금 확인

청구와 입금을 연결하고 ,
남은 금액을 확인합니다 .

부분 입금과 묶음 입금이 생기는 후불 거래에서
근거와 현재 잔액을 보고 사용자가 직접 확정합니다 .

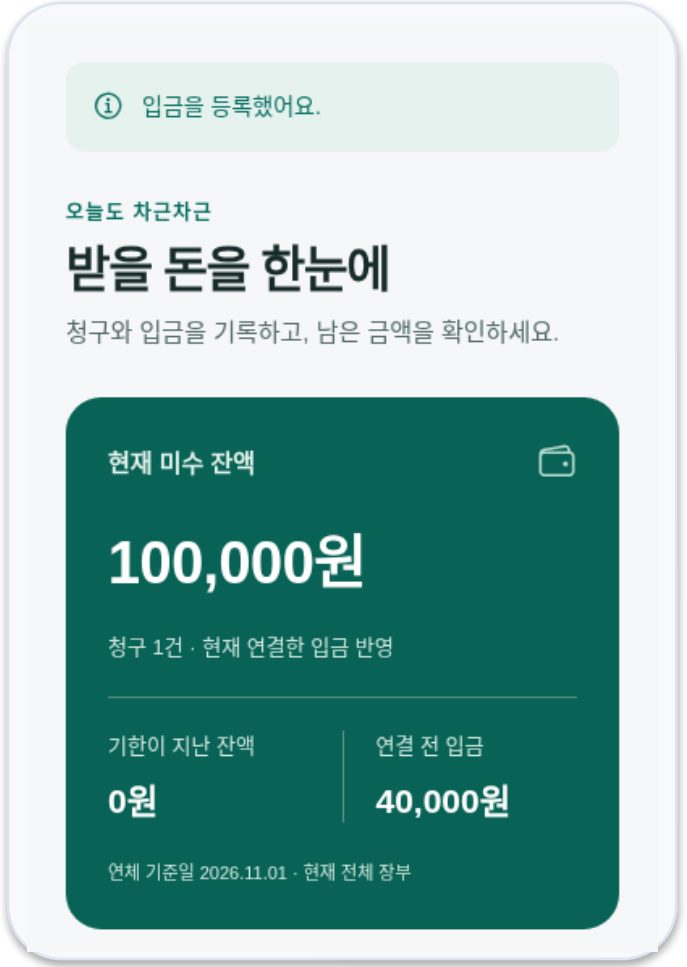
01
청구 · 입금 등록

02
연결 후보 검토

03
직접 확정 · 취소

서버 Java 21 · Spring Boot · PostgreSQL · Flyway

앱 React Native · Expo · TypeScript



PROJECT 02

아키텍처와 금융 도메인

화면은 입력 · 검토를 , 서버와 DB 는 소유권 · 금액 · 확정을 담당합니다 .

React Native · Expo
입력 · 검토 · 사용자 확정

→

Java · Spring Boot
인증 · 권한 · 잔액 검사

→

PostgreSQL · Flyway
원본 · 배분 · 취소 이력

청구와 입금 사이에 배분 원장을 둡니다 .

청구 Invoice

입금 Payment

→

배분 Allocation
부분 · 묶음 · 분할 연결

→

활성 배분 → 현재 잔액

취소 이력 추가 → 잔액 복구

PROJECT 02

문제 해결 | 동시 요청과 응답 유실

한 입금을 초과 배분하거나 , 같은 요청이 중복 저장되지 않도록 설계했습니다 .

동시 요청의 초과 배분 방지

01 입금 → 청구 , 같은 순서로 잠금

02 현재 잔액과 요청 합계 재검사

03 가용액 초과 시 전체 롤백

응답을 받지 못해도 같은 요청으로

Idempotency-Key + 정규화 요청 해시

실제 저장 후 응답만 503 으로 변경
탭 왕복 후 같은 키로 다시 확정

요청 2 회 ➡ 배분 1 개

PROJECT 02

AI 에이전트를 어떻게 활용했는가

업무 규칙과 검증 기준을 지시하고 , 역할 · 사용량 · 개발 순서를 조율했습니다 .

01

업무 기준을 먼저 지정

“금액 계산은 기존 Java 로직을 사용한다 .”

기존 코드 재사용 · FREE 우선

AI 에는 조회 도구만 허용

02

역할과 사용량 제어

“동시 실행은 최대 2 개로 제한하세요 .”

총괄 → 분석 · 개발 · 독립 검토

문서는 경량 , 코딩은 Sol

03

실제 실행 후 개선 요청

“실제 브라우저에서 주요 상호작용을 확인한다 .”

DB· 브라우저 결과 보고 요청

발견된 오류 수정 · 재검증

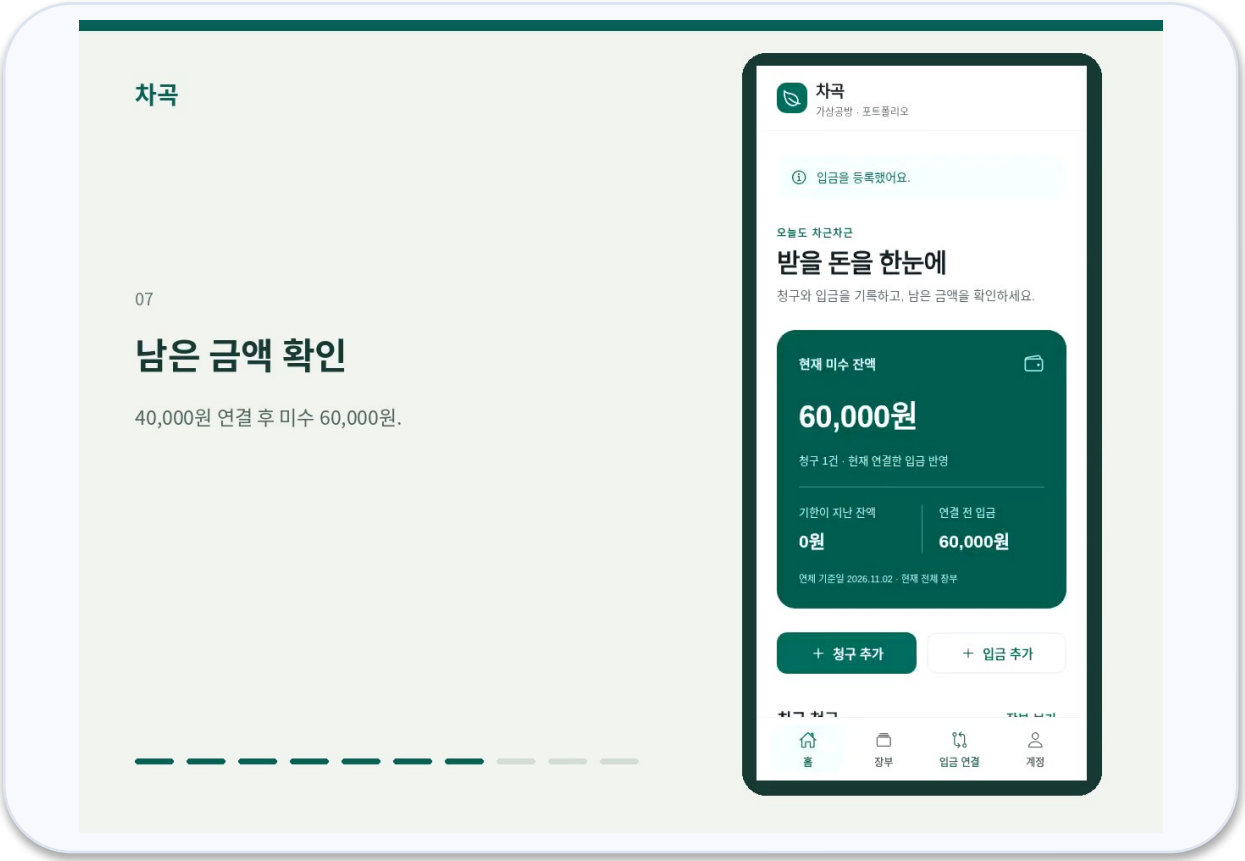
검토에서 발견한 오류를 다시 검증

공백 수정 → 같은 요청 키 유지 / 지연 응답 → 최신 조회 유지

PROJECT 02

시연과 검증

청구 등록부터 부분 연결 · 완납 · 취소 후 잔액 복구까지 실제 API 로 확인했습니다 .



검증 결과

186 개
백엔드 테스트

13 개
클라이언트 테스트

9 개 + 9 개
실제 브라우저 · API 흐름 / 기존 화면

시연 영상 열기 · 1 분 46 초